

Міністерство освіти та науки України

Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»

Факультет інформатики та обчислювальної техніки

Кафедра інформатики та програмної інженерії

Звіт до комп’ютерного практикуму №1 з дисципліни «Комп’ютерне моделювання дискретно-подійних систем»

Прийняв:
Викладач
Д.С. Коринецький

Виконав
Студент 4 курсу, гр. ІП-33
Ломакін Максим

Оцінка: _____

«___» _____ 2026 р.

Київ 2026

Генератори випадкових чисел

1. Текст завдання:

- ✓ Згенерувати 10000 випадкових чисел двома вказаними нижче способами. **50 балів.**
  - Згенерувати випадкове число за формулою $x_i = -(1/\lambda)\cdot \ln \xi_i$, де ξ_i – випадкове число, рівномірно розподілене в інтервалі (0;1). Числа ξ_i можна створювати за допомогою вбудованого в мову програмування генератора випадкових чисел. Побудувати гістограму частот, знайти середнє і дисперсію цих випадкових чисел, сформувати гіпотезу про закон розподілу. Перевірити на відповідність експоненційному закону розподілу $F(x) = 1 - e^{-\lambda x}$ за критерієм згоди χ^2. Перевірку зробити при різних значеннях λ.
  - Згенерувати випадкове число за формулою $z_{i+1} = a\cdot z_i \pmod c$, $x_{i+1} = z_{i+1}/c$, де $a = 5^{13}$, $c = 2^{31}$. Побудувати гістограму частот, знайти середнє і дисперсію цих випадкових чисел, сформувати гіпотезу про закон розподілу. Перевірити на відповідність рівномірному закону розподілу в інтервалі (0;1) за критерієм згоди χ^2. Перевірку зробити при різних значеннях параметрів a і c.
  - Зробити висновки щодо запропонованих способів генерування випадкових величин.
- ✓ Дослідити часову затримку програми на виконання методу сортування колекції 1000000 об’єктів. Виконати ідентифікацію закону розподілу за зібраними даними. Створити генератор випадкових чисел, який імітує часову затримку програми на виконання методу сортування колекції 1000000 об’єктів. Зробити висновки щодо точності відтворення часової затримки в імітаційному алгоритмі. **50 балів.**

2. Код програми:

Програму написано мовою Python 3.12 з бібліотеками NumPy, SciPy та Matplotlib. Вихідний код, виміряні дані та отримані графіки розміщено в репозиторії https://github.com/kqjqxb/Computer-simulation (каталог `lab1`). Програма складається з чотирьох модулів:

- `stats_utils.py` — статистичні функції: вибіркові характеристики, критерій згоди χ^2 з об'єднанням інтервалів, у які потрапило менше 5 значень, перевірка кореляції (формули 1.6–1.7) та критерій серій (формула 1.4);
- `task1_generators.py` — завдання 1: експоненційний генератор, мультиплікативний конгруентний генератор, обчислення періоду та перевірки;
- `measure_sorting.py` — завдання 2: вимірювання часу сортування та запис замірів у файл `data/sort_times.csv`;
- `task2_analysis.py` — завдання 2: ідентифікація закону розподілу та імітаційні генератори затримки.

Файл stats_utils.py

```
import numpy as np
from scipy import stats


def describe(data):
    data = np.asarray(data, dtype=float)
    return {
        "n": data.size,
        "mean": data.mean(),
        "var": data.var(ddof=1),
        "std": data.std(ddof=1),
        "min": data.min(),
        "max": data.max(),
        "median": float(np.median(data)),
        "skew": float(stats.skew(data, bias=False)),
    }


def merge_small_bins(observed, expected, edges, min_count=5):
    """Об'єднує сусідні інтервали, поки в кожному очікувана й спостережувана кількість не стане
    не меншою за min_count."""
    observed, expected, edges = list(observed), list(expected), list(edges)
    i = 0
    while i < len(observed) and len(observed) > 1:
        if observed[i] >= min_count and expected[i] >= min_count:
            i += 1
            continue
        j = i + 1 if i + 1 < len(observed) else i - 1
        lo, hi = min(i, j), max(i, j)
        observed[lo] += observed[hi]
        expected[lo] += expected[hi]
        del observed[hi], expected[hi], edges[hi]
        i = lo
    return np.array(observed), np.array(expected), np.array(edges)


def chi_square(data, cdf, n_params=0, bins=20, edges=None, alpha=0.05, open_tails=True):
    """Критерій згоди χ² (формула 1.29) для неперервного закону з функцією розподілу cdf.

    Інтервали за замовчуванням рівні на [min; max] (формули 1.20-1.21). При open_tails крайні інтервали
    розширюються до меж носія закону, щоб сума теоретичних імовірностей дорівнювала 1.
    """
    data = np.asarray(data, dtype=float)
    n = data.size
    if edges is None:
        edges = np.linspace(data.min(), data.max(), bins + 1)
    edges = np.asarray(edges, dtype=float)
    observed, _ = np.histogram(data, bins=edges)

    cdf_values = cdf(edges)
```

```
    if open_tails:
        cdf_values[0], cdf_values[-1] = 0.0, 1.0
    expected = n * np.diff(cdf_values)

    observed, expected, merged_edges = merge_small_bins(observed, expected, edges)
    k = observed.size
    statistic = float(np.sum((observed - expected) ** 2 / expected))
    dof = k - 1 - n_params
    critical = float(stats.chi2.ppf(1 - alpha, dof))
    return {
        "chi2": statistic,
        "critical": critical,
        "dof": dof,
        "k": k,
        "p_value": float(stats.chi2.sf(statistic, dof)),
        "accepted": statistic < critical,
        "observed": observed,
        "expected": expected,
        "edges": merged_edges,
    }


def correlation_test(z, max_lag=10):
    """Перевірка незалежності за формулами (1.6)-(1.7): ρ_j та A_j = ρ_j / sqrt(D(ρ_j))."""
    z = np.asarray(z, dtype=float)
    rows = []
    for j in range(1, max_lag + 1):
        h = z.size - j
        rho = 12.0 / h * np.dot(z[:h], z[j:]) - 3.0
        a = rho / np.sqrt((13 * h - 6) / h ** 2)
        rows.append((j, rho, a))
    return rows


RUNS_A = np.array([
    [4529.4, 9044.9, 13568, 18091, 22615, 27892],
    [9044.9, 18097, 27139, 36187, 45234, 55789],
    [13568, 27139, 40721, 54281, 67852, 83685],
    [18091, 36187, 54281, 72414, 90470, 111580],
    [22615, 45234, 67852, 90470, 113262, 139476],
    [27892, 55789, 83685, 111580, 139476, 172860],
])
RUNS_B = np.array([1 / 6, 5 / 24, 11 / 120, 19 / 720, 29 / 5040, 1 / 840])


def runs_up_test(z):
    """Критерій серій (формула 1.4): довжини зростаючих серій, R порівнюється з χ²(6; 0.9) = 10.6."""
    z = np.asarray(z, dtype=float)
    counts = np.zeros(6)
    length = 1
    for prev, cur in zip(z[:-1], z[1:]):
        if cur > prev:
            length += 1
        else:
            counts[min(length, 6) - 1] += 1
            length = 1
    counts[min(length, 6) - 1] += 1
    n = z.size
    d = counts - n * RUNS_B
    r = float(d @ RUNS_A @ d / n)
    return {"runs": counts.astype(int), "R": r, "critical": 10.6, "accepted": r < 10.6}
```

Файл task1_generators.py

```
"""Завдання 1: експоненційний генератор (метод оберненої функції)
та мультиплікативний конгруентний генератор."""
import json
import math
import random
from pathlib import Path

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
from scipy import stats
```

```python
from stats_utils import chi_square, correlation_test, describe, runs_up_test

N = 10_000
BINS = 20
SEED = 2026
REPEATS = 200
RESULTS = Path(__file__).resolve().parent / "results"

LAMBDAS = [0.5, 1.0, 2.0, 5.0]

LCG_PARAMS = [
    ("5^13", 5 ** 13, "2^31", 2 ** 31, "параметри завдання"),
    ("5^13", 5 ** 13, "2^40", 2 ** 40, "більший модуль"),
    ("5^17", 5 ** 17, "2^48", 2 ** 48, "більші a і c"),
    ("65539", 65539, "2^31", 2 ** 31, "RANDU"),
    ("5", 5, "2^31", 2 ** 31, "малий множник"),
    ("5^13", 5 ** 13, "2^12", 2 ** 12, "малий модуль"),
    ("6", 6, "2^31", 2 ** 31, "парний множник"),
]


def tex(label):
    base, _, power = label.partition("^")
    return f"${base}^{{{power}}}$" if power else f"${base}$"


def exponential_sample(lam, n, rng):
    # 1 - random() лежить у (0; 1], тож логарифм завжди визначений
    return np.array([-math.log(1.0 - rng.random()) / lam for _ in range(n)])


def lcg(a, c, z0, n):
    z = z0
    out = np.empty(n)
    for i in range(n):
        z = a * z % c
        out[i] = z / c
    return out


def lcg_period(a, c, z0, limit=10 ** 7):
    """Довжина періоду послідовності z_i. Для непарного a і c = 2^B період дорівнює порядку a за модулем c."""
    is_power_of_two = c & (c - 1) == 0
    if a % 2 == 1 and z0 % 2 == 1 and is_power_of_two:
        order, power = 1, a % c
        while power != 1:
            power = power * power % c
            order *= 2
        return order
    seen = {}
    z = z0
    for i in range(limit):
        if z in seen:
            return i - seen[z]
        seen[z] = i
        z = a * z % c
    return None


def exponential_part(rng):
    rows = []
    fig, axes = plt.subplots(2, 2, figsize=(10, 7))
    for lam, ax in zip(LAMBDAS, axes.flat):
        x = exponential_sample(lam, N, rng)
        d = describe(x)
        lam_hat = 1 / d["mean"]
        test = chi_square(x, lambda t: stats.expon.cdf(t, scale=1 / lam_hat), n_params=1, bins=BINS)

        accepted = 0
        for _ in range(REPEATS):
            y = exponential_sample(lam, N, rng)
            repeat = chi_square(y, lambda t: stats.expon.cdf(t, scale=y.mean()), n_params=1, bins=BINS)
            accepted += repeat["accepted"]

        rows.append({
            "lambda": lam, "mean": d["mean"], "var": d["var"],
            "theory_mean": 1 / lam, "theory_var": 1 / lam ** 2,
```

```
            "lambda_hat": lam_hat, "chi2": test["chi2"], "critical": test["critical"], "dof": test["dof"],
            "k": test["k"], "p_value": test["p_value"], "accepted": bool(test["accepted"]),
            "accept_rate": accepted / REPEATS,
            "table": [[float(test["edges"][i]), float(test["edges"][i + 1]), int(o), float(e)]
                      for i, (o, e) in enumerate(zip(test["observed"], test["expected"]))],
        })

        counts, edges, _ = ax.hist(x, bins=BINS, color="#7da7d9", edgecolor="black", linewidth=0.6,
                                   label="спостережувані частоти")
        centers = (edges[:-1] + edges[1:]) / 2
        ax.plot(centers, N * np.diff(stats.expon.cdf(edges, scale=1 / lam_hat)), "o-", color="#c0392b",
                markersize=3.5, linewidth=1.4, label=f"$np_i^T$, λ^= {lam_hat:.4f}")
        ax.set_title(f"λ = {lam}")
        ax.set_xlabel("x")
        ax.set_ylabel("кількість влучень $n_i$")
        ax.grid(alpha=0.3)
        ax.legend(fontsize=8)
    fig.tight_layout()
    fig.savefig(RESULTS / "task1_exponential_hist.png", dpi=200)
    plt.close(fig)
    return rows


def lcg_part(rng):
    rows = []
    samples = {}
    for a_label, a, c_label, c, note in LCG_PARAMS:
        z0 = 1
        x = lcg(a, c, z0, N)
        samples[(a_label, c_label)] = x
        d = describe(x)
        test = chi_square(x, lambda t: np.clip(t, 0, 1), bins=BINS, edges=np.linspace(0, 1, BINS + 1))
        corr = correlation_test(x)
        runs = runs_up_test(x)
        period = lcg_period(a, c, z0)

        accepted = np.zeros(3)
        for _ in range(REPEATS):
            y = lcg(a, c, rng.randrange(1, c, 2), N)
            accepted += [
                chi_square(y, lambda t: np.clip(t, 0, 1), edges=np.linspace(0, 1, BINS + 1))["accepted"],
                all(abs(A) < 1.645 for _, _, A in correlation_test(y)),
                runs_up_test(y)["accepted"],
            ]
        rates = accepted / REPEATS

        rows.append({
            "a": a_label, "c": c_label, "note": note, "mean": d["mean"], "var": d["var"],
            "distinct": int(np.unique(x).size), "period": period,
            "chi2": test["chi2"], "critical": test["critical"], "dof": test["dof"], "k": test["k"],
            "p_value": test["p_value"], "accepted": bool(test["accepted"]), "accept_rate": rates[0],
            "corr_rate": rates[1], "runs_rate": rates[2],
            "max_abs_A": float(max(abs(r[2]) for r in corr)),
            "corr": [[j, float(r), float(A)] for j, r, A in corr],
            "runs_R": runs["R"], "runs_ok": bool(runs["accepted"]), "runs": runs["runs"].tolist(),
            "counts": [int(v) for v in np.histogram(x, bins=np.linspace(0, 1, BINS + 1))[0]],
        })

    fig, axes = plt.subplots(2, 2, figsize=(10, 7))
    shown = [("5^13", "2^31"), ("5^13", "2^40"), ("5^17", "2^48"), ("65539", "2^31")]
    for (a_label, c_label), ax in zip(shown, axes.flat):
        ax.hist(samples[(a_label, c_label)], bins=np.linspace(0, 1, BINS + 1), color="#8fcf9f",
                edgecolor="black", linewidth=0.6)
        ax.axhline(N / BINS, color="#c0392b", linestyle="--", linewidth=1.4, label="N/m = 500")
        ax.set_ylim(0, 650)
        ax.set_title(f"a = {tex(a_label)}, c = {tex(c_label)}")
        ax.set_xlabel("x")
        ax.set_ylabel("кількість влучень $n_i$")
        ax.grid(alpha=0.3)
        ax.legend(fontsize=8, loc="lower right")
    fig.tight_layout()
    fig.savefig(RESULTS / "task1_lcg_hist.png", dpi=200)
    plt.close(fig)

    fig, axes = plt.subplots(1, 3, figsize=(12, 4.2))
    for (a_label, c_label), ax in zip([("5^13", "2^31"), ("5", "2^31"), ("5^13", "2^12")], axes):
        x = samples[(a_label, c_label)]
```

```
        ax.scatter(x[:-1], x[1:], s=0.6, color="#2c5d8f")
        ax.set_title(f"a = {tex(a_label)}, c = {tex(c_label)}")
        ax.set_xlabel("$x_i$")
        ax.set_ylabel("$x_{i+1}$")
        ax.set_aspect("equal")
    fig.tight_layout()
    fig.savefig(RESULTS / "task1_lcg_pairs.png", dpi=200)
    plt.close(fig)

    fig, axes = plt.subplots(1, 2, figsize=(11, 4))
    for (a_label, c_label), ax in zip([("5^13", "2^31"), ("65539", "2^31")], axes):
        x = samples[(a_label, c_label)]
        s = 9 * x[:-2] - 6 * x[1:-1] + x[2:]
        ax.scatter(x[:-2], s, s=0.6, color="#2c5d8f")
        ax.set_title(f"a = {tex(a_label)}, c = {tex(c_label)}")
        ax.set_xlabel("$x_i$")
        ax.set_ylabel("$9x_i - 6x_{i+1} + x_{i+2}$")
        ax.grid(alpha=0.3)
    fig.tight_layout()
    fig.savefig(RESULTS / "task1_lcg_triples.png", dpi=200)
    plt.close(fig)
    return rows


def print_report(exp_rows, lcg_rows):
    print(f"ЕКСПОНЕНЦІЙНИЙ ГЕНЕРАТОР x = -ln(ξ)/λ, N = {N}, α = 0.05")
    print(f"{'λ':>5} {'середнє':>9} {'1/λ':>7} {'дисперсія':>10} {'1/λ²':>7} {'λ*':>8} "
          f"{'k':>3} {'ст.в.':>5} {'χ²':>8} {'χ²кр':>7} {'p':>6}  рішення   частка прийнять")
    for r in exp_rows:
        print(f"{r['lambda']:>5} {r['mean']:>9.5f} {r['theory_mean']:>7.4f} {r['var']:>10.5f} "
              f"{r['theory_var']:>7.4f} {r['lambda_hat']:>8.5f} {r['k']:>3} {r['dof']:>5} {r['chi2']:>8.3f} "
              f"{r['critical']:>7.3f} {r['p_value']:>6.3f}  {'прийнято' if r['accepted'] else 'відхилено':9} "
              f"{r['accept_rate']:.3f}")

    print(f"\nКОНГРУЕНТНИЙ ГЕНЕРАТОР z(i+1) = a·z(i) mod c, x = z/c, z0 = 1, N = {N}, m = {BINS}, α = 0.05")
    print(f"{'a':>6} {'c':>5} {'середнє':>9} {'дисперсія':>10} {'різних':>7} {'період':>12} "
          f"{'χ²':>9} {'χ²кр':>7} {'p':>6}  {'рішення':9} {'max|A|':>8} {'R':>9}  "
          "частка прийнять χ²/кор./серії")
    for r in lcg_rows:
        print(f"{r['a']:>6} {r['c']:>5} {r['mean']:>9.5f} {r['var']:>10.5f} {r['distinct']:>7} "
              f"{str(r['period']):>12} {r['chi2']:>9.2f} {r['critical']:>7.3f} {r['p_value']:>6.3f}  "
              f"{'прийнято' if r['accepted'] else 'відхилено':9} {r['max_abs_A']:>8.2f} {r['runs_R']:>9.2f}  "
              f"{r['accept_rate']:.3f}/{r['corr_rate']:.3f}/{r['runs_rate']:.3f}")
    print("Теоретично: середнє 0.5, дисперсія 1/12 = 0.08333; |A_j| < 1.645 (j = 1..10); R < 10.6")
    print(f"Частки прийнять обчислено за {REPEATS} вибірками з різними непарними z0")


def main():
    RESULTS.mkdir(exist_ok=True)
    rng = random.Random(SEED)
    exp_rows = exponential_part(rng)
    lcg_rows = lcg_part(rng)
    print_report(exp_rows, lcg_rows)
    with open(RESULTS / "task1.json", "w", encoding="utf-8") as f:
        json.dump({"exponential": exp_rows, "lcg": lcg_rows}, f, ensure_ascii=False, indent=1)


if __name__ == "__main__":
    main()
```

Файл measure_sorting.py

```
"""Завдання 2, крок 1: вимірювання часу сортування колекції з 1 000 000 об'єктів."""
import csv
import gc
import platform
import random
import sys
import time
from operator import attrgetter
from pathlib import Path

SIZE = 1_000_000
RUNS = 500
SEED = 2026
OUT = Path(__file__).resolve().parent / "data" / "sort_times.csv"
```

```
class Order:
    __slots__ = ("order_id", "amount")

    def __init__(self, order_id, amount):
        self.order_id = order_id
        self.amount = amount


def main():
    rng = random.Random(SEED)
    orders = [Order(i, rng.randint(0, 10 ** 9)) for i in range(SIZE)]
    key = attrgetter("amount")

    warmup = orders[:]
    warmup.sort(key=key)
    del warmup

    OUT.parent.mkdir(exist_ok=True)
    with open(OUT, "w", newline="") as f:
        writer = csv.writer(f)
        writer.writerow(["run", "seconds"])
        for run in range(1, RUNS + 1):
            collection = orders[:]
            rng.shuffle(collection)
            gc.collect()

            start = time.perf_counter()
            collection.sort(key=key)
            elapsed = time.perf_counter() - start

            writer.writerow([run, f"{elapsed:.7f}"])
            del collection
            if run % 50 == 0:
                f.flush()
                print(f"{run}/{RUNS}: {elapsed:.4f} c", flush=True)

    cpu = platform.processor() or platform.machine()
    print(f"Python {sys.version.split()[0]}, {platform.platform()}, {cpu}")
    print(f"Збережено: {OUT}")


if __name__ == "__main__":
    main()
```

Файл task2_analysis.py

```
"""Завдання 2, крок 2: ідентифікація закону розподілу часу сортування та імітаційні генератори."""
import csv
import json
import random
from pathlib import Path

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
from scipy import stats

from stats_utils import chi_square, describe

HERE = Path(__file__).resolve().parent
DATA = HERE / "data" / "sort_times.csv"
RESULTS = HERE / "results"
BINS = 20
GENERATED = 10_000
SEED = 7

CANDIDATES = [
    ("нормальний", stats.norm),
    ("логнормальний зі зсувом", stats.lognorm),
    ("гамма (Ерланга) зі зсувом", stats.gamma),
    ("Вейбулла зі зсувом", stats.weibull_min),
]
TABLE_PROBS = np.unique(np.concatenate([np.linspace(0, 1, 21), [0.97, 0.98, 0.99, 0.995]]))
```

```
def load_times():
    with open(DATA, newline="") as f:
        return np.array([float(row["seconds"]) for row in csv.DictReader(f)])


def fit(dist, t):
    if dist is stats.norm:
        return dist.fit(t)
    # початкові наближення: зсув трохи нижче мінімуму, інакше MLE часто збігається до виродженого розв'язку
    loc = t.min() - 0.5 * t.std()
    if dist is stats.lognorm:
        return dist.fit(t, 0.8, loc=loc, scale=np.median(t) - loc)
    return dist.fit(t, 1.0, loc=loc, scale=t.std())


def identify(t):
    rows = []
    for name, dist in CANDIDATES:
        params = fit(dist, t)
        test = chi_square(t, lambda x, d=dist, p=params: d.cdf(x, *p), n_params=len(params), bins=BINS)
        rows.append({"law": name, "dist": dist, "params": params, **test})
    return rows


class TableGenerator:
    """Табличний метод (формули 1.13-1.14): кусково-лінійна апроксимація емпіричної функції розподілу."""

    def __init__(self, sample, probs=TABLE_PROBS):
        self.a = np.asarray(probs)
        self.x = np.quantile(sample, self.a)

    def __call__(self, rng):
        z = rng.random()
        i = int(np.searchsorted(self.a, z, side="left"))
        i = min(max(i, 1), len(self.a) - 1)
        return self.x[i - 1] + (self.x[i] - self.x[i - 1]) / (self.a[i] - self.a[i - 1]) * (z - self.a[i - 1])

    def cdf(self, t):
        return np.interp(t, self.x, self.a, left=0.0, right=1.0)


class InverseGenerator:
    """Метод оберненої функції: r = F^(-1)(ξ) для підібраного параметричного закону."""

    def __init__(self, dist, params):
        self.dist, self.params = dist, params

    def __call__(self, rng):
        return float(self.dist.ppf(1.0 - rng.random(), *self.params))

    def cdf(self, t):
        return self.dist.cdf(t, *self.params)


def generate(generator, n, seed):
    rng = random.Random(seed)
    return np.array([generator(rng) for _ in range(n)])


def homogeneity(real, simulated, bins=BINS):
    """Двовибірковий χ² (однорідність) на спільних інтервалах
    та двовибірковий критерій Колмогорова-Смірнова."""
    edges = np.quantile(real, np.linspace(0, 1, bins + 1))
    edges[0], edges[-1] = -np.inf, np.inf
    obs_real = np.histogram(real, bins=edges)[0]
    obs_sim = np.histogram(simulated, bins=edges)[0]
    table = np.vstack([obs_real, obs_sim])
    table = table[:, table.sum(axis=0) > 0]
    chi2, p, dof, _ = stats.chi2_contingency(table)
    ks = stats.ks_2samp(real, simulated)
    return {"chi2": float(chi2), "dof": int(dof), "critical": float(stats.chi2.ppf(0.95, dof)), "p": float(p),
            "ks": float(ks.statistic), "ks_p": float(ks.pvalue)}


def accuracy(real, simulated):
    def q(x, p):
        return float(np.quantile(x, p))
```

```
    metrics = [("середнє", np.mean), ("ст. відхилення", lambda x: np.std(x, ddof=1)),
               ("мінімум", np.min), ("медіана", lambda x: q(x, 0.5)), ("90 %-квантиль", lambda x: q(x, 0.9)),
               ("99 %-квантиль", lambda x: q(x, 0.99))]
    return [(name, float(f(real)), float(f(simulated)), abs(f(simulated) - f(real)) / f(real) * 100)
            for name, f in metrics]


def plot_series(t):
    fig, ax = plt.subplots(figsize=(10, 3.4))
    ax.plot(np.arange(1, t.size + 1), t * 1000, ".", markersize=3, color="#2c5d8f")
    ax.axhline(t.mean() * 1000, color="#c0392b", linewidth=1.2, label=f"середнє {t.mean() * 1000:.1f} мс")
    ax.set_xlabel("номер прогону")
    ax.set_ylabel("час сортування, мс")
    ax.grid(alpha=0.3)
    ax.legend()
    fig.tight_layout()
    fig.savefig(RESULTS / "task2_series.png", dpi=200)
    plt.close(fig)


def plot_histogram(t, rows):
    fig, ax = plt.subplots(figsize=(10, 4.8))
    counts, edges, _ = ax.hist(t * 1000, bins=BINS, color="#7da7d9", edgecolor="black", linewidth=0.6,
                               label="спостережувані частоти $n_i$")
    centers = (edges[:-1] + edges[1:]) / 2 / 1000
    colors = ["#7f8c8d", "#c0392b", "#27ae60", "#8e44ad"]
    for row, color in zip(rows, colors):
        expected = t.size * np.diff(row["dist"].cdf(edges / 1000, *row["params"]))
        ax.plot(centers * 1000, expected, "o-", markersize=3, linewidth=1.3, color=color, label=row["law"])
    ax.set_xlabel("час сортування, мс")
    ax.set_ylabel("кількість влучень")
    ax.grid(alpha=0.3)
    ax.legend(fontsize=9)
    fig.tight_layout()
    fig.savefig(RESULTS / "task2_histogram.png", dpi=200)
    plt.close(fig)


def plot_comparison(t, simulated):
    lo, hi = t.min(), np.quantile(t, 0.995)
    edges = np.linspace(lo, hi, 31)
    fig, axes = plt.subplots(1, 2, figsize=(11, 4.2))
    ax = axes[0]
    ax.hist(t * 1000, bins=edges * 1000, density=True, alpha=0.55, color="#2c5d8f", label="реальні заміри")
    for label, sample, color in simulated:
        ax.hist(sample * 1000, bins=edges * 1000, density=True, histtype="step", linewidth=1.6, color=color,
                label=label)
    ax.set_xlabel("час сортування, мс")
    ax.set_ylabel("відносна частота / мс")
    ax.grid(alpha=0.3)
    ax.legend(fontsize=8)

    ax = axes[1]
    ax.plot(np.sort(t) * 1000, np.arange(1, t.size + 1) / t.size, color="#2c5d8f", linewidth=2.2,
            label="реальні заміри")
    for label, sample, color in simulated:
        ecdf = np.arange(1, sample.size + 1) / sample.size
        ax.plot(np.sort(sample) * 1000, ecdf, color=color, linewidth=1.2, label=label)
    ax.set_xlim(lo * 1000, hi * 1000)
    ax.set_xlabel("час сортування, мс")
    ax.set_ylabel("емпірична функція розподілу")
    ax.grid(alpha=0.3)
    ax.legend(fontsize=8, loc="lower right")
    fig.tight_layout()
    fig.savefig(RESULTS / "task2_comparison.png", dpi=200)
    plt.close(fig)


def main():
    RESULTS.mkdir(exist_ok=True)
    t = load_times()
    d = describe(t)
    print(f"ЧАС СОРТУВАННЯ 1 000 000 ОБ'ЄКТІВ: n = {d['n']}")
    print(f"середнє = {d['mean'] * 1000:.3f} мс, ст. відхилення = {d['std'] * 1000:.3f} мс, "
          f"дисперсія = {d['var'] * 1e6:.4f} мс², медіана = {d['median'] * 1000:.3f} мс")
    print(f"мінімум = {d['min'] * 1000:.3f} мс, максимум = {d['max'] * 1000:.3f} мс, "
```

```
      f"асиметрія = {d['skew']:.3f}, "
      f"коефіцієнт варіації = {d['std'] / d['mean'] * 100:.2f} %")

rows = identify(t)
print(f"\nПЕРЕВІРКА ГІПОТЕЗ (χ², {BINS} інтервалів до об'єднання, α = 0.05)")
for r in rows:
    params = ", ".join(f"{v:.6g}" for v in r["params"])
    print(f"{r['law']:28} параметри ({params}); k = {r['k']}, ст.в. = {r['dof']}, χ² = {r['chi2']:.2f}, "
          f"χ²кр = {r['critical']:.2f}, p = {r['p_value']:.4f} -> "
          f"{'прийнято' if r['accepted'] else 'відхилено'}")
best = max(rows, key=lambda r: r["p_value"])
print(f"Найкраща відповідність: {best['law']}")

plot_series(t)
plot_histogram(t, rows)

table_gen = TableGenerator(t)
inverse_gen = InverseGenerator(best["dist"], best["params"])
generators = [("табличний метод", table_gen, "#c0392b"), (f"{best['law']}", inverse_gen, "#27ae60")]

print("\nТАБЛИЦЯ ТАБЛИЧНОГО ГЕНЕРАТОРА (a_i = F(x_i), x_i у мс):")
print("  ".join(f"{a:.3f}:{x * 1000:.2f}" for a, x in zip(table_gen.a, table_gen.x)))

simulated, evaluation = [], []
for i, (label, gen, color) in enumerate(generators):
    sample = generate(gen, GENERATED, SEED + i)
    simulated.append((label, sample, color))
    hom = homogeneity(t, sample)
    acc = accuracy(t, sample)
    self_test = chi_square(t, gen.cdf, bins=BINS)
    evaluation.append({"label": label, "homogeneity": hom, "accuracy": acc,
                       "fit_chi2": self_test["chi2"], "fit_critical": self_test["critical"],
                       "fit_dof": self_test["dof"]})
    print(f"\nГЕНЕРАТОР «{label}», згенеровано {GENERATED} значень:")
    print(f"{'характеристика':16} {'реальні, мс':>12} {'імітація, мс':>13} {'похибка, %':>11}")
    for name, real_v, sim_v, err in acc:
        print(f"{name:16} {real_v * 1000:>12.3f} {sim_v * 1000:>13.3f} {err:>11.2f}")
    print(f"χ² однорідності = {hom['chi2']:.2f} (ст.в. {hom['dof']}), χ²кр = {hom['critical']:.2f}, "
          f"p = {hom['p']:.3f}); КС: D = {hom['ks']:.4f}, p = {hom['ks_p']:.3f}")

first, second = t[: t.size // 2], t[t.size // 2:]
drift = stats.mannwhitneyu(first, second)
drift_ks = stats.ks_2samp(first, second)
print(f"\nСТАЦІОНАРНІСТЬ: прогони 1-{first.size}: середнє {first.mean() * 1000:.2f} мс, "
      f"ст. відх. {first.std(ddof=1) * 1000:.2f} мс; прогони {first.size + 1}-{t.size}: середнє "
      f"{second.mean() * 1000:.2f} мс, ст. відх. {second.std(ddof=1) * 1000:.2f} мс")
print(f"Манна-Вітні p = {drift.pvalue:.2e}, КС p = {drift_ks.pvalue:.2e}")

train, test = t[0::2], t[1::2]
print(f"\nПЕРЕХРЕСНА ПЕРЕВІРКА: побудова за непарними прогонами ({train.size}), "
      f"порівняння з парними ({test.size})")
best_dist = best["dist"]
cross = []
for label, gen in [("табличний метод", TableGenerator(train)),
                   (best["law"], InverseGenerator(best_dist, fit(best_dist, train)))]:
    sample = generate(gen, GENERATED, SEED + 10)
    hom = homogeneity(test, sample)
    err = abs(sample.mean() - test.mean()) / test.mean() * 100
    cross.append({"label": label, "homogeneity": hom, "mean_error": err})
    print(f"{label:28} похибка середнього = {err:.2f} %, χ² = {hom['chi2']:.2f} "
          f"(χ²кр = {hom['critical']:.2f}, "
          f"p = {hom['p']:.3f}), КС p = {hom['ks_p']:.3f}")

plot_comparison(t, simulated)

with open(RESULTS / "task2.json", "w", encoding="utf-8") as f:
    json.dump({
        "describe": {k: float(v) for k, v in d.items()},
        "laws": [{"law": r["law"], "params": [float(v) for v in r["params"]], "chi2": r["chi2"],
                  "critical": r["critical"], "dof": r["dof"], "k": r["k"], "p": r["p_value"],
                  "accepted": bool(r["accepted"])} for r in rows],
        "best": best["law"],
        "table": [[float(a), float(x)] for a, x in zip(table_gen.a, table_gen.x)],
        "evaluation": evaluation, "cross": cross,
        "drift": {"first_mean": float(first.mean()), "first_std": float(first.std(ddof=1)),
                  "second_mean": float(second.mean()), "second_std": float(second.std(ddof=1)),
                  "mw_p": float(drift.pvalue), "ks_p": float(drift_ks.pvalue)},
```

```
        }, f, ensure_ascii=False, indent=1)

if __name__ == "__main__":
    main()
```

3. Результати роботи програми:

Завдання 1:

Для кожного генератора отримано вибірку з N = 10000 чисел, побудовано гістограму частот, обчислено вибіркове середнє та незміщену вибіркову дисперсію:

$$\bar{x} = (1/n)\cdot\Sigma x_i, \quad S^2 = (1/(n-1))\cdot\Sigma(x_i - \bar{x})^2.$$

Відповідність гіпотетичному закону перевірялась критерієм згоди Пірсона на рівні значущості α = 0,05:

$$\chi^2 = \Sigma\,(n_i - np_i^T)^2 / np_i^T, \quad p_i^T = F(x_i) - F(x_{i-1}),$$

де n_i – спостережувана кількість влучень в і-й інтервал, np_i^T – очікувана за теоретичним законом. Інтервали будуються рівними на [min; max] (20 інтервалів), сусідні інтервали з кількістю влучень меншою за 5 об'єднуються. Кількість ступенів вільності дорівнює k − 1 − r, де k – кількість інтервалів після об'єднання, r – кількість параметрів закону, оцінених за вибіркою. Гіпотеза приймається, якщо $\chi^2 < \chi^2_{кр}$.

Крім одиночної перевірки, для кожного набору параметрів критерій повторено на 200 незалежних вибірках. Якщо генератор справді відтворює закон, гіпотеза має прийматися приблизно в 1 − α = 95 % випадків: помітно менша частка означає систематичне відхилення, а частка, близька до 100 %, – «занадто рівномірну» (невипадкову) послідовність.

1) Експоненційний генератор (метод оберненої функції)

Числа ξ_i створювались вбудованим генератором Python `random.random()` (вихор Мерсенна). Оскільки він повертає значення з [0; 1), використовувалось 1 − ξ, що лежить у (0; 1] і не дає ln 0. Для експоненційного закону μ = 1/λ, $\sigma^2 = 1/\lambda^2$; параметр закону для перевірки оцінювався за вибіркою як $\hat{\lambda} = 1/\bar{x}$ (формула 1.26), тому r = 1. Досліджено λ = 0,5; 1; 2; 5.

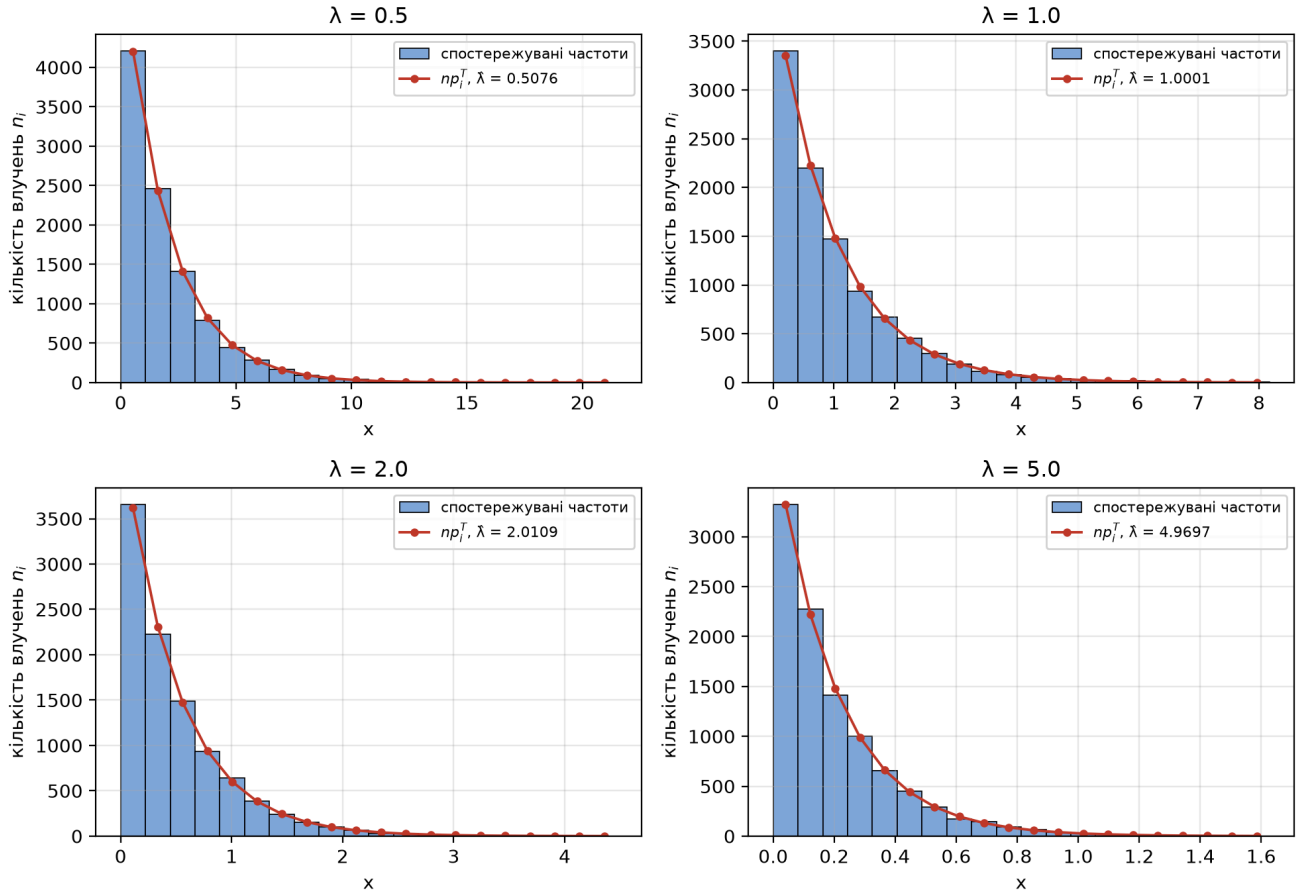


Рисунок 1 – Гістограми частот експоненційного генератора та теоретично очікувані кількості влучень

Гістограми (рис. 1) мають максимум біля нуля й монотонно спадають праворуч, а середнє приблизно дорівнює середньому квадратичному відхиленню, тож формуємо гіпотезу про експоненційний закон. Результати перевірки наведено в таблиці 1.

Таблиця 1

Характеристики експоненційного генератора і перевірка за критерієм χ^2

λ	**$\bar{x}$**	**1/λ**	**S^2**	**$1/\lambda^2$**	**$\hat{\lambda}$**	**k / ст.в.**	**χ^2**	**$\chi^2_{кр}$**	**Гіпотеза**	**Прийнято з 200**
0,5	1,9702	2,0000	4,0052	4,0000	0,5076	14 / 12	8,39	21,03	прийнято	96,5 %
1,0	0,9999	1,0000	1,0199	1,0000	1,0001	18 / 16	17,15	26,30	прийнято	94,0 %
2,0	0,4973	0,5000	0,2446	0,2500	2,0109	16 / 14	10,51	23,68	прийнято	96,0 %
5,0	0,2012	0,2000	0,0399	0,0400	4,9697	17 / 15	13,72	25,00	прийнято	94,0 %

Таблиця 2

Розрахунок критерію χ^2 для $\lambda = 1$ (після об’єднання інтервалів)

Інтервал	$\mathbf{n_i}$	$\mathbf{np_i^T}$	$\mathbf{(n_i - np_i^T)^2/np_i^T}$
[0,000; 0,409)	3402	3354,4	0,674
[0,409; 0,817)	2202	2228,7	0,320
[0,817; 1,226)	1472	1481,3	0,058
[1,226; 1,634)	939	984,5	2,103
[1,634; 2,043)	670	654,3	0,375
[2,043; 2,451)	453	434,9	0,754
[2,451; 2,860)	296	289,0	0,167
[2,860; 3,268)	188	192,1	0,088
[3,268; 3,677)	118	127,7	0,734
[3,677; 4,085)	78	84,9	0,555
[4,085; 4,494)	56	56,4	0,003
[4,494; 4,902)	40	37,5	0,169
[4,902; 5,311)	24	24,9	0,034
[5,311; 5,719)	17	16,6	0,012
[5,719; 6,127)	21	11,0	9,076
[6,127; 6,536)	11	7,3	1,857
[6,536; 7,353)	7	8,1	0,148
[7,353; ∞)	6	6,4	0,025
Σ	10000	10000	17,151

У таблиці 2 наведено розрахунок для $\lambda = 1$: останні інтервали гістограми, в які потрапило менше 5 значень, об’єднано, а крайній правий інтервал розширено до $+\infty$, щоб сума теоретичних імовірностей дорівнювала 1. Отримано $k = 18$, кількість ступенів вільності $18 - 1 - 1 = 16$, $\chi^2 = 17{,}15 < \chi^2_{кр} = 26{,}30$.

Для всіх значень λ вибіркові середнє та дисперсія відрізняються від теоретичних $1/\lambda$ та $1/\lambda^2$ не більше ніж на 1,5 %, що відповідає статистичній похибці вибірки з 10000 значень (стандартна похибка середнього $\sigma/\sqrt{N} = 1$ %). Зі збільшенням λ розподіл стискається до нуля, а середнє та дисперсія зменшуються пропорційно $1/\lambda$ та $1/\lambda^2$. Гіпотеза про експоненційний закон прийнята для всіх λ, а частка прийнять на 200 повторах (94–96,5 %) збігається з очікуваними 95 %, тобто розбіжності між гістограмою і теоретичним законом мають лише випадковий характер.

2) Мультиплікативний конгруентний генератор

Числа генерувались за формулою $z_{i+1} = a \cdot z_i \pmod c$, $x_{i+1} = z_{i+1}/c$ з початковим значенням $z_0 = 1$. Обчислення виконуються з цілими числами Python довільної довжини, тому добуток $a \cdot z_i$ не переповнюється. Для перевірки рівномірності інтервал (0; 1) поділено на m = 20 рівних частин з очікуваною кількістю влучень N/m = 500; параметри закону не оцінюються, тож ступенів вільності m − 1 = 19, $\chi^2_{кр} = 30{,}14$. Теоретично для рівномірного закону на (0; 1) M = 1/2 = 0,5, D = 1/12 ≈ 0,08333.

Крім параметрів завдання досліджено ще шість наборів (a, c): з більшим модулем, з більшими a і c, відомий генератор RANDU ($a = 65539 = 2^{16} + 3$, $c = 2^{31}$), малий множник a = 5, малий модуль $c = 2^{12}$ та парний множник a = 6, який порушує умову $a = 1 + 4k$. Період послідовності для непарних a і $c = 2^B$ обчислювався як мультиплікативний порядок a за модулем c (для $a \equiv 3$ або 5 (mod 8) він дорівнює 2^{B-2}), в інших випадках – безпосереднім пошуком першого повторення значення z_i.

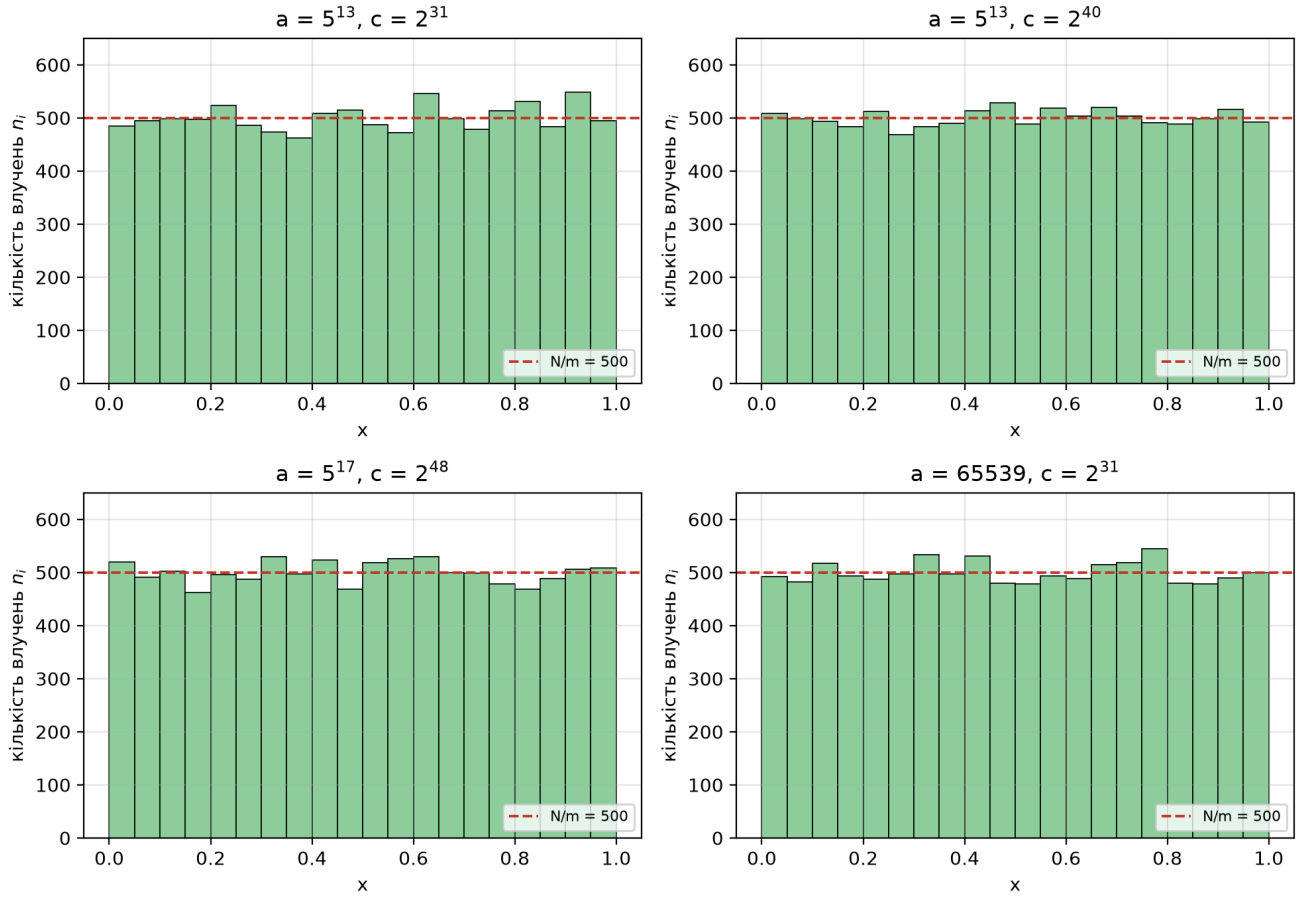


Рисунок 2 – Гістограми частот конгруентного генератора

Гістограми (рис. 2) мають стовпчики приблизно однакової висоти навколо N/m = 500, тож формуємо гіпотезу про рівномірний закон розподілу на (0; 1). Результати перевірки наведено в таблиці 3.

Таблиця 3

Характеристики конгруентного генератора і перевірка на рівномірність ($z_0 = 1$, χ^2кр = 30,14)

a	**c**	**Примітка**	**$\bar{x}$**	**S^2**	**Різних значень**	**Період**	**χ^2**	**Гіпотеза**
5^{13}	2^{31}	параметри завдання	0,5041	0,08385	10000	2^{29}	21,69	прийнято
5^{13}	2^{40}	більший модуль	0,5012	0,08304	10000	2^{38}	8,68	прийнято
5^{17}	2^{48}	більші a і c	0,4996	0,08303	10000	2^{46}	16,42	прийнято
65539	2^{31}	RANDU	0,5002	0,08290	10000	2^{29}	15,16	прийнято
5	2^{31}	малий множник	0,4970	0,08528	10000	2^{29}	32,82	відхилено
5^{13}	2^{12}	малий модуль	0,4996	0,08336	1024	2^{10}	0,97	прийнято
6	2^{31}	парний множник	0,0008	0,00050	31	1	189280,68	відхилено

Для параметрів завдання $a = 5^{13}$, $c = 2^{31}$ отримано $\bar{x} = 0{,}5041$, $S^2 = 0{,}08385$, $\chi^2 = 21{,}69 < 30{,}14$ – гіпотеза про рівномірний закон приймається. Період генератора $2^{29} = 536870912$ чисел, тобто вибірка з 10000 значень займає менше 0,002 % періоду і не містить повторів.

Критерій χ^2 перевіряє лише рівномірність, тому для всіх наборів додатково виконано перевірку незалежності: перевірку кореляції послідовностей, зсунутих на $j = 1\ldots10$ (формули 1.6–1.7, умова $|A_j| < 1{,}645$ для всіх j), та критерій серій зростання (формула 1.4, умова $R < 10{,}6$). Обидва критерії мають свій рівень похибки першого роду, тому, як і для χ^2, підраховано частку прийнять на 200 вибірках з різними непарними z_0. Для еталону ті самі частки отримано для генератора NumPy (PCG64): χ^2 – 95 %, кореляція (10 лагів одночасно) – близько 79 %, серії – близько 90 %.

Таблиця 4

Перевірка незалежності та частки прийнять гіпотез на 200 вибірках

a	**c**	**max$\|A_j\|$ ($z_0 = 1$)**	**R ($z_0 = 1$)**	**χ^2**	**Кореляція**	**Серії**
5^{13}	2^{31}	1,89	6,75	95,0 %	76,5 %	90,5 %
5^{13}	2^{40}	0,92	20,79	96,5 %	80,0 %	91,5 %
5^{17}	2^{48}	0,60	9,91	94,5 %	80,0 %	90,5 %
65539	2^{31}	0,62	17,03	95,0 %	72,5 %	84,5 %
5	2^{31}	4,80	615,91	90,5 %	0,0 %	0,0 %
5^{13}	2^{12}	1,16	343,04	100,0 %	100,0 %	0,0 %
6	2^{31}	83,12	54848,86	0,0 %	0,0 %	0,0 %

Таблиця 5

Перевірка кореляції для $a = 5^{13}$, $c = 2^{31}$, $z_0 = 1$

j	**ρ_j**	**A_j**
1	0,0449	1,245
2	0,0681	1,889
3	0,0558	1,547
4	0,0515	1,427
5	0,0669	1,856
6	0,0610	1,691
7	0,0624	1,730
8	0,0468	1,298
9	0,0435	1,206
10	0,0513	1,421

Таблиця 5 показує, що для $z_0 = 1$ у базового генератора одне значення $|A_j|$ трохи перевищує 1,645. Це не свідчить про залежність: межа 1,645 відповідає 10 % імовірності хибного відхилення для кожного j, тож при одночасній перевірці десяти зсувів хоча б одне перевищення трапляється приблизно у кожній п’ятій вибірці навіть для ідеального генератора. Частки прийнять у таблиці 4 для $a = 5^{13}$, $c = 2^{31}$ (95 %, 76,5 %, 90,5 %) збігаються з еталонними.

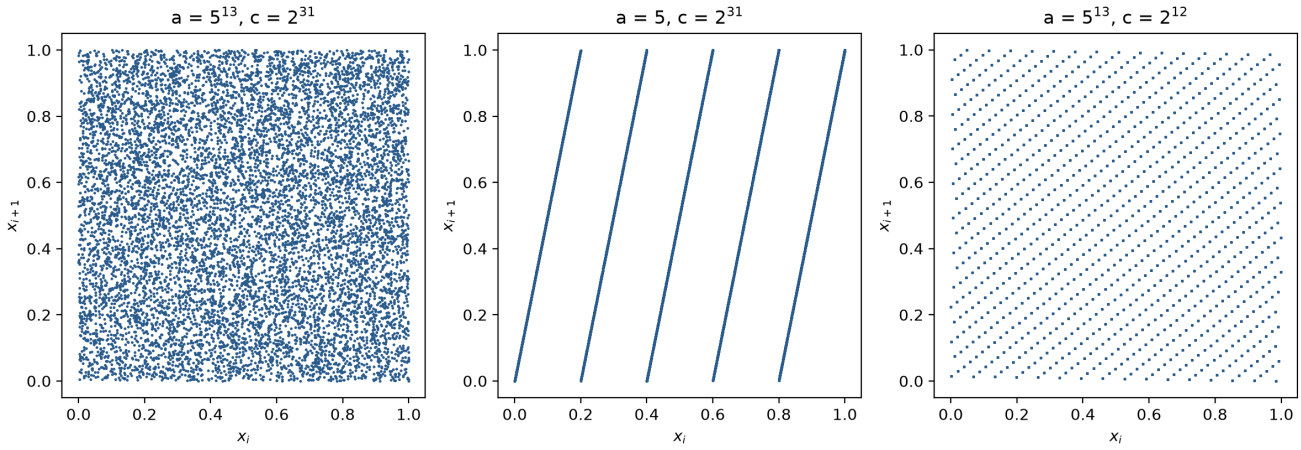


Рисунок 3 – Пари сусідніх чисел (x_i, x_{i+1}) в одиничному квадраті

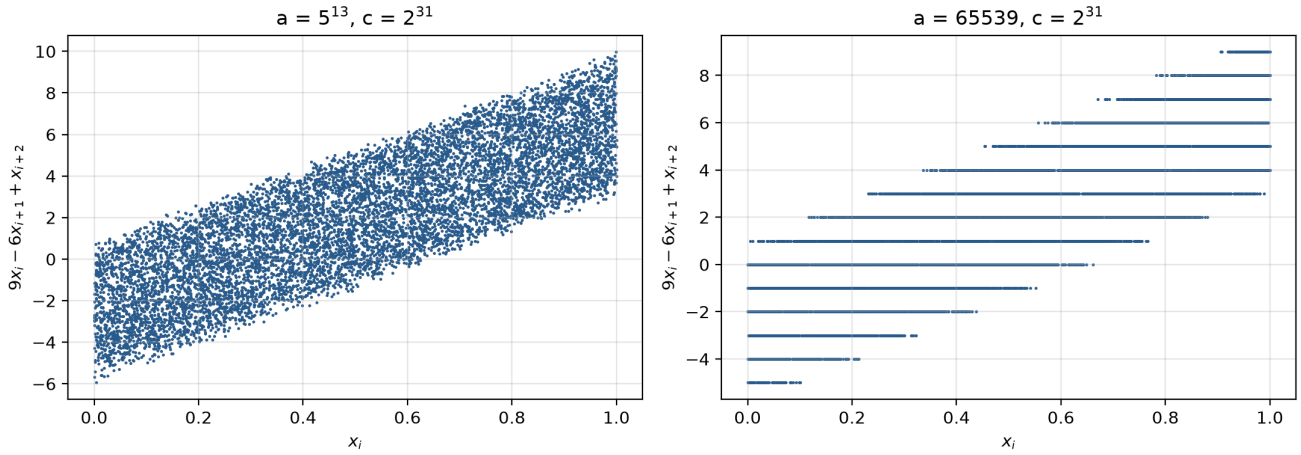


Рисунок 4 – Проєкція трійок (x_i, x_{i+1}, x_{i+2}) на напрям (9; −6; 1)

Теоретичний тест із зображенням пар сусідніх чисел (рис. 3) підтверджує результати: для $a = 5^{13}$, $c = 2^{31}$ точки рівномірно заповнюють квадрат, для $a = 5$ усі пари лежать на п’яти прямих $x_{i+1} = 5x_i - k$, а для $c = 2^{12}$ утворюють регулярну ґратку з 1024 вузлів.

Аналіз окремих наборів параметрів:

- $a = 5^{13}$ з модулями 2^{40} та $a = 5^{17}$, $c = 2^{48}$: період зростає до 2^{38} і 2^{46}, крок дискретизації $1/c$ зменшується; усі перевірки проходять з тією ж частотою, що й еталон. На вибірці з 10000 чисел якість практично не відрізняється від базового варіанта – збільшення c важливе для довгих імітацій, де потрібно більше 2^{29} чисел.
- RANDU ($a = 65539$): $\chi^2 = 15{,}16$, гіпотеза про рівномірність приймається, а кореляція та серії лише трохи гірші за еталон (72,5 % і 84,5 %). Проте

оскільки $a^2 \equiv 6a - 9 \pmod{2^{31}}$, кожна трійка задовольняє $9x_i - 6x_{i+1} + x_{i+2}$ = ціле число, і всі точки в одиничному кубі лежать лише на 15 площинах (рис. 4, праворуч), тоді як для $a = 5^{13}$ ця величина неперервна (ліворуч). Тобто одновимірні тести не виявляють дефекту, який стає критичним, коли генератор використовують для кількох координат одночасно.

- $a = 5$: період максимальний (2^{29}), середнє 0,4970 і дисперсія 0,08528 близькі до теоретичних, $\chi^2 = 32{,}82$ трохи перевищує критичне значення. Однак кореляція з лагом 1 становить близько $1/a = 0{,}2$ ($\max|A_j| = 4{,}80$), $R = 615{,}9$, і на 200 вибірках перевірки кореляції та серій не пройдено жодного разу: числа рівномірні, але залежні. Крім того, після $z_0 = 1$ перші 13 значень ($5^k/2^{31}$) дуже малі.
- $c = 2^{12}$: період лише $2^{10} = 1024$, тому 10000 чисел – це майже десять повторів одного циклу (1024 різних значень). Значення $\chi^2 = 0{,}97$ підозріло мале, гіпотеза приймається в 100 % випадків замість 95 % – послідовність «занадто рівномірна», а критерій серій відхиляє її завжди ($R = 343{,}0$).
- $a = 6$ (парний): кожне множення додає множник 2 до z_i, і вже після 31 кроку $z_i = 0$ назавжди; маємо 31 різних значень, $\bar{x} = 0{,}00084$, $\chi^2 = 189281$ – генератор повністю непридатний. Це ілюструє вимогу непарності a ($a = 1 + 4k$) та z_0.

Завдання 2:

Об’єктом дослідження є виклик стандартного методу сортування `list.sort` (алгоритм Timsort) для списку з 1000000 об’єктів класу `Order` з полями `order_id` та `amount`; ключ сортування – поле `amount` (випадкове ціле від 0 до 10^9). Об’єкти створюються один раз, а перед кожним вимірюванням копія списку перемішується, тож кожен прогін сортує нову випадкову перестановку. Перед вимірюваннями виконується один прогрівальний прогін, а перед кожним заміром викликається збирач сміття, щоб він не спрацював під час сортування через залишки попереднього прогону. Час вимірюється функцією `time.perf_counter()` і охоплює лише виклик `sort`. Виконано $n = 500$ прогонів (Python 3.12, Apple M1, 8 ГБ ОЗП, macOS) у звичайному режимі роботи комп’ютера, без закриття інших програм; заміри збережено у файлі `data/sort_times.csv`.

1) Вибіркові характеристики та стаціонарність

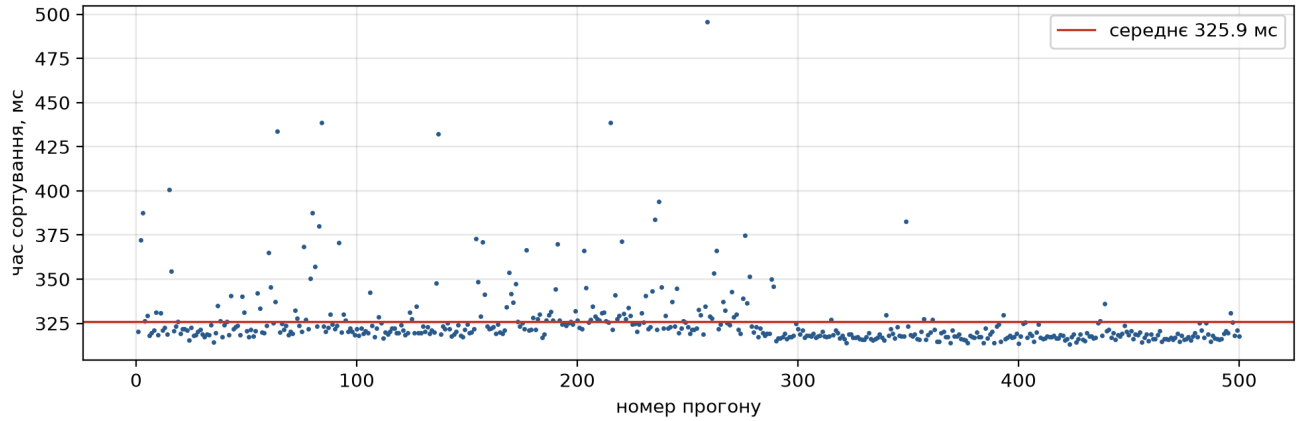


Рисунок 5 – Час сортування в послідовних прогонах

Вибіркові характеристики: $\bar{x}$ = 325,87 мс, S = 17,89 мс, медіана 320,53 мс, мінімум 313,35 мс, максимум 495,65 мс, коефіцієнт асиметрії 4,61, коефіцієнт варіації 5,5 %. Розкид невеликий порівняно із середнім, але розподіл сильно скошений праворуч: медіана менша за середнє, а максимум перевищує мінімум у 1,6 раза.

На рис. 5 видно, що величина затримки складається з двох частин. Основна маса замірів щільно лежить біля 315–325 мс – це власний час роботи алгоритму, що трохи змінюється від перестановки до перестановки. Окремі заміри на 20–180 мс більші – це прогони, під час яких процес сортування витісняла операційна система (інші програми, фонові служби, перемикання ядер процесора). Частота таких викидів змінювалась у часі: у перших 250 прогонах $\bar{x}$ = 330,50 мс, S = 20,09 мс, в останніх 250 – $\bar{x}$ = 321,24 мс, S = 13,96 мс. Двовибірковий критерій Колмогорова–Смірнова для цих половин дає $p \approx 3 \cdot 10^{-31}$, тобто умови роботи комп’ютера під час вимірювань не були стаціонарними. Тому для перевірки генератора на незалежних даних вибірку поділено не на хронологічні половини, а на непарні та парні прогони, які виміряно в однакових умовах (див. нижче).

2) Ідентифікація закону розподілу

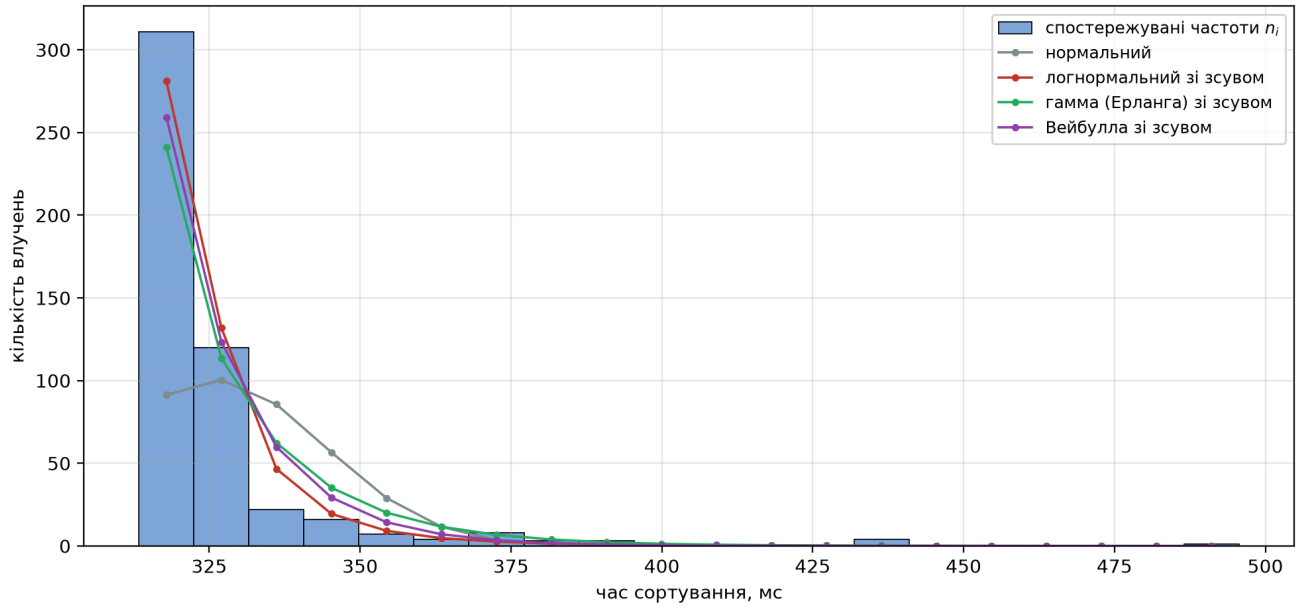


Рисунок 6 – Гістограма частот часу сортування та очікувані частоти підібраних законів

Гістограма (рис. 6, 20 рівних інтервалів на [min; max]) має різкий лівий край біля мінімального часу, максимум у першому інтервалі та довгий правий хвіст. Таку форму мають закони з нижньою межею (зсувом) і правою асиметрією з таблиці 1.1: логнормальний, Ерланга (гамма), Вейбулла. Нормальний закон перевірено для порівняння. Параметри законів оцінено методом максимальної правдоподібності; для трипараметричних законів третім параметром є зсув – мінімально можливий час сортування. Інтервали, в які потрапило менше 5 значень (весь правий хвіст), об’єднано, тому після об’єднання залишається 6–7 інтервалів. Результати наведено в таблиці 6.

Таблиця 6

Перевірка гіпотез про закон розподілу часу сортування ($\alpha = 0{,}05$)

Закон	**Параметри**	**k / ст.в.**	χ^2	$\chi^2_{кр}$	**Гіпотеза**
нормальний	$\mu = 325{,}87$ мс; $\sigma = 17{,}88$ мс	6 / 3	146,77	7,81	відхилено
логнормальний зі зсувом	$\sigma = 0{,}869$; зсув = 313,08 мс; $e^{\mu} = 8{,}19$ мс	6 / 2	30,77	5,99	відхилено
гамма (Ерланга) зі зсувом	$k = 0{,}853$; зсув = 313,35 мс; $1/\lambda = 17{,}50$ мс	7 / 3	68,66	7,81	відхилено
Вейбулла зі зсувом	$a = 0{,}987$; зсув = 313,35 мс; $b = 12{,}52$ мс	6 / 2	51,38	5,99	відхилено

Жодна з гіпотез не приймається. Нормальний закон не відтворює асиметрії ($\chi^2 = 146{,}8$). Найближчим є логнормальний закон зі зсувом ($\chi^2 = 30{,}77$ при $\chi^2_{кр} = 5{,}99$): він правильно описує нижню межу та загальну форму, але недооцінює висоту піку й важкість хвоста – реальні рідкісні затримки в 400–500 мс за ним майже неможливі. Причина в тому, що спостережувана величина є сумішшю двох механізмів (час алгоритму та випадкові витіснення процесу операційною системою), а не результатом одного процесу, який описує стандартний закон. Перевірка окремо для прогонів 291–500, де викидів майже немає, також відхиляє всі чотири закони.

Отже, закон розподілу затримки ідентифікується як емпіричний. Для генерування випадкових чисел з емпіричним законом методичні вказівки рекомендують табличний метод (п. 1.2.4), тому основний генератор побудовано саме ним. Для порівняння побудовано також генератор найкращого параметричного закону – логнормального зі зсувом – методом оберненої функції $r = F^{-1}(\xi)$.

3) Імітаційний генератор затримки

Табличний генератор використовує кусково-лінійну апроксимацію емпіричної функції розподілу, задану парами (x_i, a_i), де $a_i = F(x_i)$, а x_i – вибіркові квантилі рівня a_i. Вузли взято через 0,05 та додатково в хвості (0,97; 0,98; 0,99; 0,995), де функція розподілу змінюється найповільніше. Випадкове число обчислюється за формулою (1.14):

$$r = x_{i-1} + (x_i - x_{i-1}) / (a_i - a_{i-1}) \cdot (\xi - a_{i-1}), \text{ якщо } a_{i-1} < \xi \le a_i,$$

де ξ – рівномірно розподілене на (0; 1) число від вбудованого генератора. Отриману таблицю наведено в таблиці 7.

Таблиця 7

Таблиця емпіричної функції розподілу для табличного генератора

a_i	x_i, мс	a_i	x_i, мс	a_i	x_i, мс
0,000	313,35	0,450	319,94	0,900	339,39
0,050	315,64	0,500	320,53	0,950	354,82
0,100	316,29	0,550	321,44	0,970	371,37
0,150	316,92	0,600	322,06	0,980	382,85
0,200	317,36	0,650	323,21	0,990	401,14
0,250	317,90	0,700	324,34	0,995	436,31
0,300	318,33	0,750	325,59	1,000	495,65
0,350	318,88	0,800	327,50		
0,400	319,38	0,850	330,81		

4) Точність відтворення затримки

Кожним генератором отримано 10000 значень, які порівнювались з реальними замірами за основними характеристиками (відносна похибка $\delta = |x_{іміт} - x_{реал}| / x_{реал} \cdot 100\ \%$) та за двовибірковими критеріями однорідності: χ^2 (20 інтервалів, рівноймовірних за реальною вибіркою) та Колмогорова–Смірнова (статистика D – найбільша відстань між емпіричними функціями розподілу).

Таблиця 8

Порівняння реальних замірів і 10000 значень імітаційних генераторів

Характеристика	Реальні, мс	Табличний, мс	δ, %	Логнормальний, мс	δ, %
середнє	325,87	325,81	0,02	325,10	0,23
ст. відхилення	17,89	17,33	3,13	12,68	29,11
мінімум	313,35	313,36	0,00	313,46	0,04
медіана	320,53	320,49	0,01	321,41	0,27
90 %-квантиль	339,39	339,31	0,03	338,10	0,38
99 %-квантиль	401,14	402,68	0,38	375,32	6,44

Таблиця 9

Двовибіркові перевірки однорідності реальних і згенерованих значень ($\alpha = 0,05$)

Генератор	χ^2	$\chi^2_{кр}$	p (χ^2)	D	p (КС)
Табличний (усі 500 замірів)	1,19	30,14	1,000	0,0152	1,000
Логнормальний (усі 500 замірів)	52,38	30,14	0,000	0,0680	0,023
Табличний (перехресна)	17,43	30,14	0,560	0,0457	0,672
Логнормальний (перехресна)	38,11	30,14	0,006	0,1037	0,010

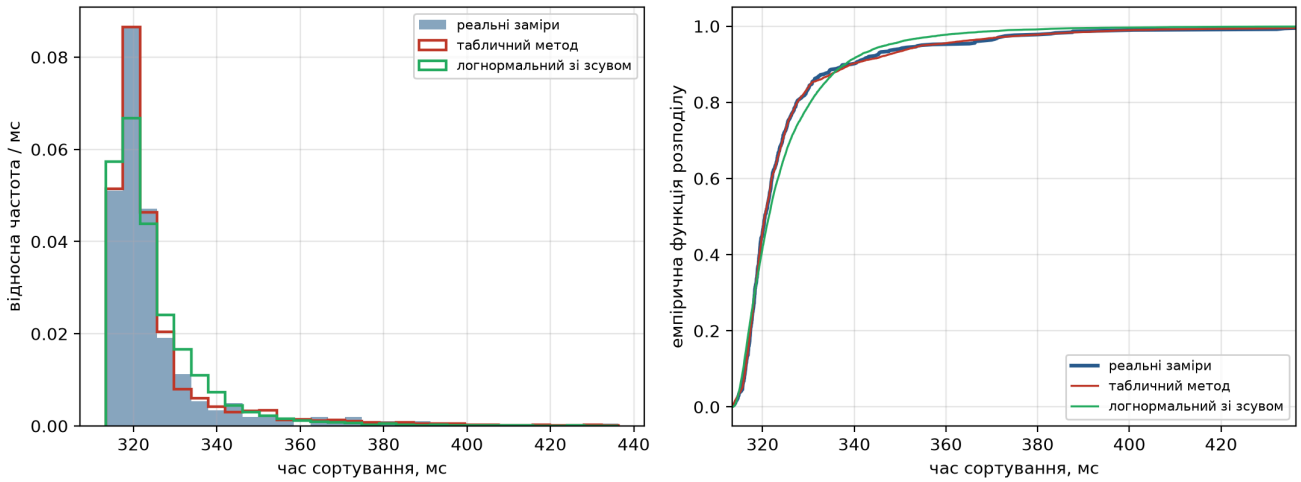


Рисунок 7 – Реальні заміри та значення імітаційних генераторів: гістограми та емпіричні функції розподілу

Табличний генератор відтворює середнє, медіану та 90 %-квантиль з похибкою до 0,03 %, стандартне відхилення – з похибкою 3,1 %, а 99 %-квантиль – 0,38 % (табл. 8). Його емпірична функція розподілу практично збігається з реальною (рис. 7). Оскільки таблицю побудовано за тими самими 500 замірами, з якими йде порівняння, такий збіг частково очікуваний. Тому виконано перехресну перевірку: генератор побудовано лише за 250 непарними прогонами, а згенеровані числа порівнювались з 250 парними прогонами, які в побудові не брали участі. І в цьому випадку гіпотеза про однорідність приймається (χ^2 = 17,43 < 30,14, КС p = 0,67, табл. 9), а похибка середнього становить 0,51 %.

Логнормальний генератор відтворює середнє з похибкою 0,23 %, але занижує стандартне відхилення на 29 % і 99 %-квантиль на 6,4 %: він не створює рідкісних великих затримок. Двовибіркові критерії його відхиляють і на повній вибірці, і при перехресній перевірці (χ^2 = 38,11, КС p = 0,010). Отже, для імітації часу сортування слід використовувати табличний генератор.

Висновок до практикуму:

У роботі досліджено два способи генерування випадкових величин і побудовано імітаційний генератор часової затримки реальної програми.

Генератор $x = -(1/\lambda)\cdot \ln \xi$ на основі вбудованого генератора Python відтворює експоненційний закон: для λ = 0,5; 1; 2; 5 вибіркові середнє та дисперсія збігаються з $1/\lambda$ та $1/\lambda^2$ у межах статистичної похибки, гіпотеза за критерієм χ^2 приймається, а частка прийнять на 200 повторних вибірках (94–96,5 %) відповідає

рівню значущості 0,05. Метод оберненої функції точний, якщо вихідні числа ξ_i якісні; його недолік – обчислення логарифма для кожного числа.

Конгруентний генератор з $a = 5^{13}$, $c = 2^{31}$ дає числа з середнім 0,5041 та дисперсією 0,08385 (теорія 0,5 і 0,08333), проходить перевірки рівномірності, кореляції та серій з тією ж частотою, що й еталонний генератор, і має період $2^{29} \approx 5{,}4 \cdot 10^8$. Він простий і швидкий, проте якість сильно залежить від параметрів: парний множник вироджує послідовність у нуль, малий модуль дає короткий цикл і «занадто рівномірні» числа, малий множник $a = 5$ – сильну кореляцію сусідніх чисел, а RANDU проходить усі одновимірні тести, але його трійки лежать на 15 площинах. Критерій χ^2 перевіряє лише рівномірність, тому генератор потрібно додатково перевіряти на незалежність і періодичність, а результат одиночної перевірки оцінювати з урахуванням імовірності хибного відхилення.

Час сортування 1000000 об’єктів методом `list.sort` має середнє близько 326 мс, різкий нижній край (≈ 313 мс – власний час алгоритму) і довгий правий хвіст із затримками до 496 мс, спричинений роботою операційної системи; частота таких затримок змінювалась під час вимірювань. Нормальний, логнормальний, гамма (Ерланга) та Вейбулла закони за критерієм χ^2 відхилено, тож закон затримки ідентифіковано як емпіричний.

Імітаційний генератор, побудований табличним методом, відтворює середнє, медіану та 90 %-квантиль затримки з похибкою до 0,03 %, стандартне відхилення – з похибкою 3,1 %, і проходить двовибіркові критерії χ^2 та Колмогорова–Смірнова навіть на даних, які не використовувались для його побудови (похибка середнього 0,51 %). Генератор логнормального закону відтворює лише середнє, а розкид і хвіст затримок занижує. Точність імітації обмежена насамперед не генератором, а нестабільністю самої затримки: її розподіл залежить від фонового навантаження комп’ютера, тому таблицю генератора слід будувати за замірами, отриманими в умовах, для яких виконується моделювання.